1. Defining Operator Overloading

Theory

Operator Overloading is a feature of C++ that allows programmers to redefine the working of existing operators for user-defined data types (classes).

Normally operators such as +, -, *, ==, ++ work only with built-in data types like int, float, and char.

Using **operator overloading**, these operators can also work with objects.

Advantages

- Makes programs easy to understand.
- Improves code readability.
- Allows objects to behave like built-in data types.
- Reduces the need for separate functions.

Syntax

```
return_type operator symbol(parameters)
{
   // code
}
```

Example

operator +

operator -

operator *

operator ++

operator --

Practical Example

```
#include<iostream.h>
#include<conio.h>

class Number
{
   int n;

public:
   void getData()
   {
      cin>>n;
   }

   void operator -()
   {
      n=-n;
   }

   void display()
   {
      cout<<"Number = "<<n;
   }
};
```

```
void main()
{
   clrscr();

   Number obj;

   cout<<"Enter Number: ";
   obj.getData();

   -obj;

   obj.display();

   getch();
}
```

Output

```
Enter Number: 15

Number = -15
```

---

2. Operator Overloading with Member Function

Theory

In member function overloading, the operator function is declared inside the class.
The left-hand side object automatically calls the overloaded operator.

Syntax

```
class ClassName
{
public:

return_type operator symbol(arguments)
{
}
};
```

Example

```
obj1 + obj2
becomes
obj1.operator+(obj2);
```

Practical Example

```
#include<iostream.h>
#include<conio.h>

class Test
{
   int num;

public:
```

```
   void input()
   {
      cin>>num;
   }

   void operator +(Test t)
   {
      cout<<"Addition = "<<num+t.num;
   }
};

void main()
{
   clrscr();

   Test t1,t2;

   cout<<"Enter First Number: ";
   t1.input();

   cout<<"Enter Second Number: ";
   t2.input();

   t1+t2;

   getch();
}
```

Output

```
Enter First Number: 20
Enter Second Number: 30

Addition = 50
```

---

3. Unary Operator Overloading

Theory

Unary operators require only one operand.

Examples

```
++
--
-
!
```

Unary operator overloading modifies only one object.

Syntax

```
void operator ++()
{
}
```

Practical Example 1 (Unary ++)

```
#include<iostream.h>
#include<conio.h>

class Count
{
   int x;

public:

   Count()
   {
      x=10;
   }

   void operator ++()
   {
      x++;
   }

   void display()
   {
      cout<<"Value = "<<x;
   }
};

void main()
{
   clrscr();

   Count c;

   ++c;

   c.display();

   getch();
}
```

Output

Value = 11

Practical Example 2 (Unary Minus)

```
#include<iostream.h>
#include<conio.h>

class Number
{
   int n;
```

```
public:

   Number()
   {
      n=25;
   }

   void operator -()
   {
      n=-n;
   }

   void display()
   {
      cout<<n;
   }
};

void main()
{
   clrscr();

   Number obj;

   -obj;

   obj.display();

   getch();
}
```

Output

-25

---

4. Binary Operator Overloading

Theory

Binary operators require two operands.

Examples

+

-

*

/

%

==

<

>

Binary operator overloading is commonly used to perform operations between two objects.

Syntax

return_type operator +(ClassName obj)

```
{
}
```

Practical Example 1 (Addition of Two Objects)

```
#include<iostream.h>
#include<conio.h>

class Number
{
   int n;

public:

   void getData()
   {
      cin>>n;
   }

   void operator +(Number obj)
   {
      cout<<"Addition = "<<n+obj.n;
   }
};

void main()
{
   clrscr();

   Number n1,n2;

   cout<<"Enter First Number: ";
   n1.getData();

   cout<<"Enter Second Number: ";
   n2.getData();

   n1+n2;

   getch();
}
```

Output

```
Enter First Number: 40
Enter Second Number: 20

Addition = 60
```

Practical Example 2 (Complex Number Addition)

```
#include<iostream.h>
```

```
#include<conio.h>

class Complex
{
   int real,img;

public:

   void input()
   {
      cout<<"Enter Real Part: ";
      cin>>real;

      cout<<"Enter Imaginary Part: ";
      cin>>img;
   }

   void operator +(Complex c)
   {
      cout<<"Result = "<<real+c.real<<" + "<<img+c.img<<"i";
   }
};

void main()
{
   clrscr();

   Complex c1,c2;

   c1.input();

   c2.input();

   c1+c2;

   getch();
}
```

Output

```
Enter Real Part: 4
Enter Imaginary Part: 3

Enter Real Part: 6
Enter Imaginary Part: 2

Result = 10 + 5i
```